APUNTES CLASE 13-10

David Morales Vargas



Análisis Sintáctico

Devolviéndonos en el tiempo vamos a recordar al parser:

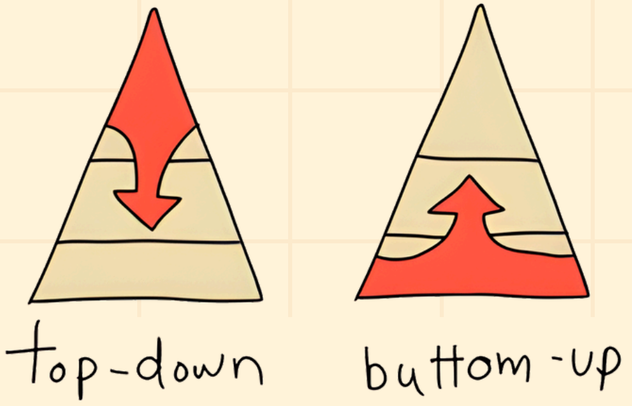
- Revisa que la sintaxis del programa esté bien.
- Verifica secuencias de tokens.
- Necesita una gramática.

Parsing

Hay muchos algoritmos para hacer parsing, cada uno tiene capacidades y propiedades diferentes.

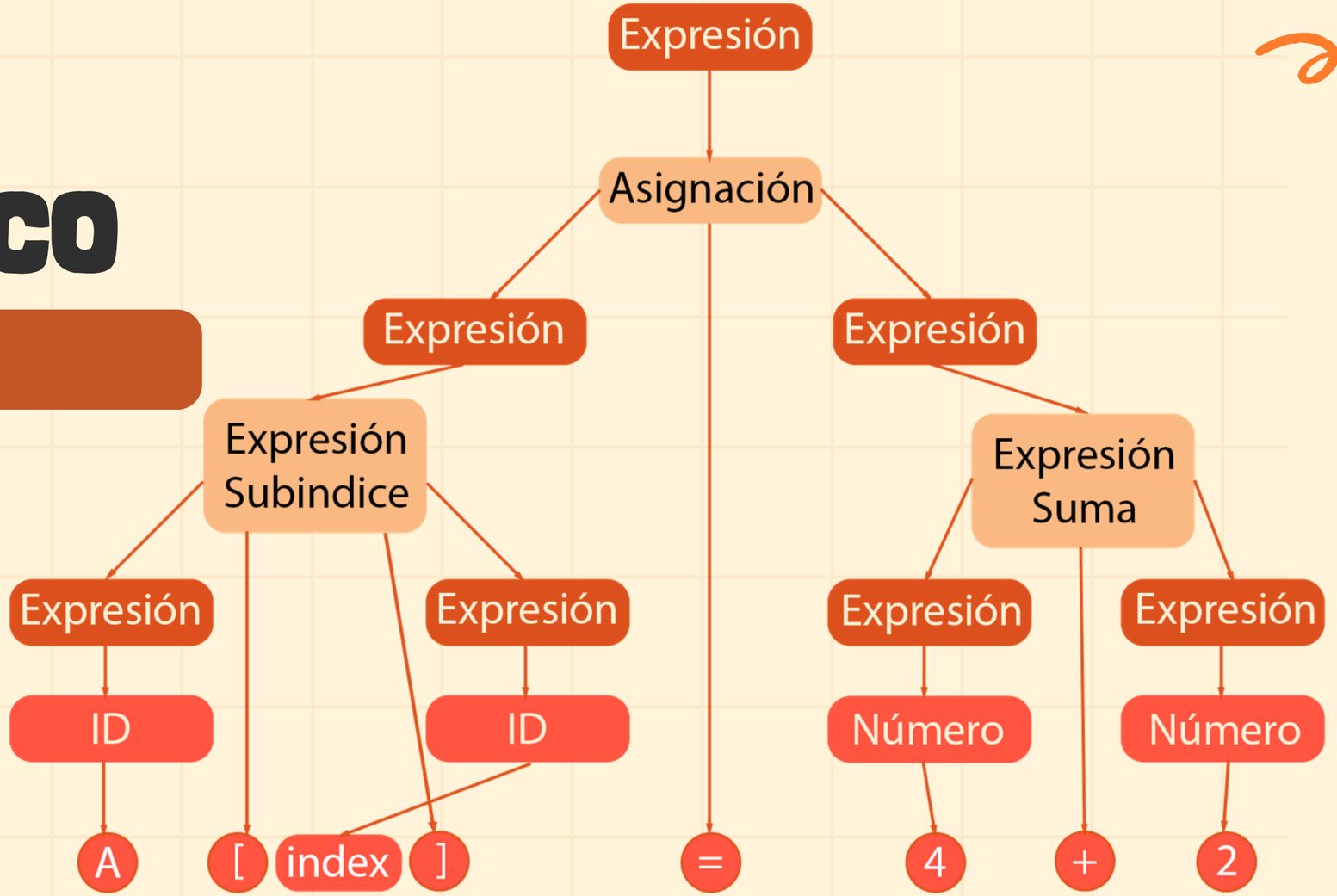
Tipos:

- Top down parsing.
- Bottom up parsing



Árbol Sintáctico

A [index] = 4 + 2



Errores de Sintaxis

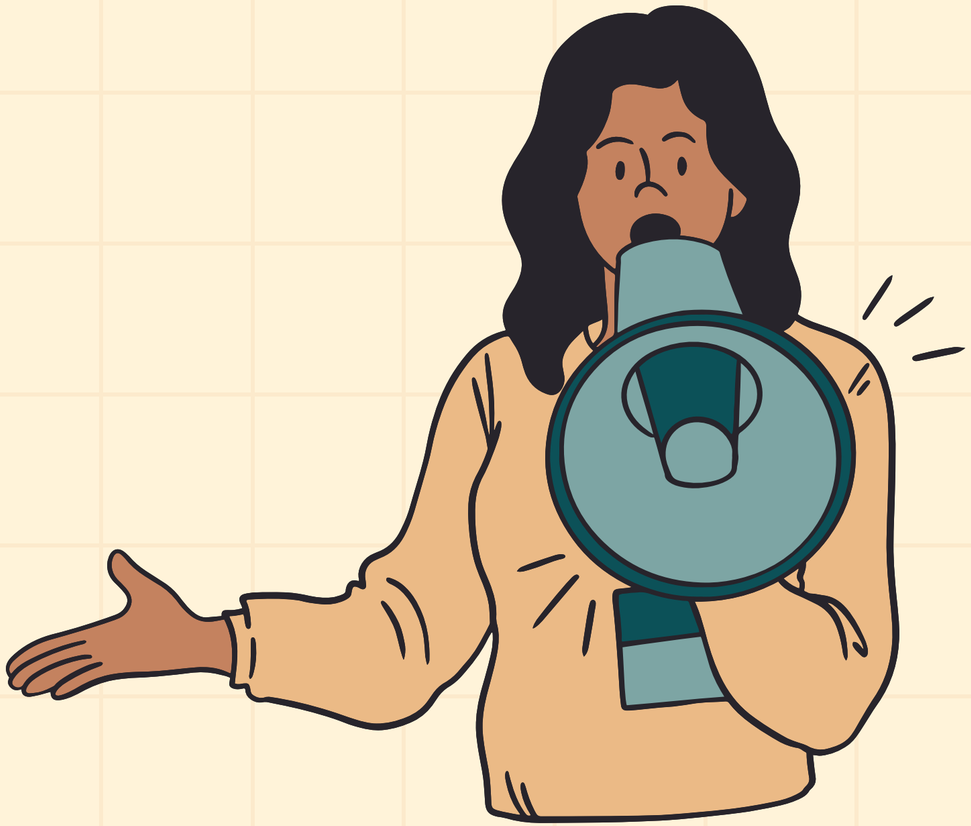
Es muy difícil manejar errores en el parser. El parser debe hacer varias cosas cuando se topa un error:

- Reportar
- Recuperar
- Reparar



Reportar

- Saber que hay un error es automático con los algoritmos de parsing.
- Pero saber exactamente que token lo está causando y donde es lo más difícil.
- También dar mensajes de error concretos es complicado, peor de los casos solo decir "Syntax Error".





Recuperar

- Un único error basta para decir que todo el programa está malo.
- Muy inutil solo reportar el primer error y dejar de ver si hay más errores, el parser tiene que saber recuperarse de un error para poder validar el resto del código.
- Para esto avanza hasta un "punto seguro", pero aquí se generan dos posibles problemas:
  - Saltar demasiado y perder código.
  - Saltar muy poco y provocar cascada de errores.



Reparar

- Todos nos preguntamos ¿Por qué el parser no nos arregla el programa si sabe lo que está malo?.
- Pero en realidad el parser si hace intentos de recuperación, los conocemos como Warnings normalmente.
- ¿Pero en realidad queremos que el parser "adivine" lo que no nosotros queriamos hacer?, así como la IA de VSCode



Traducción Dirigida por Sintaxis

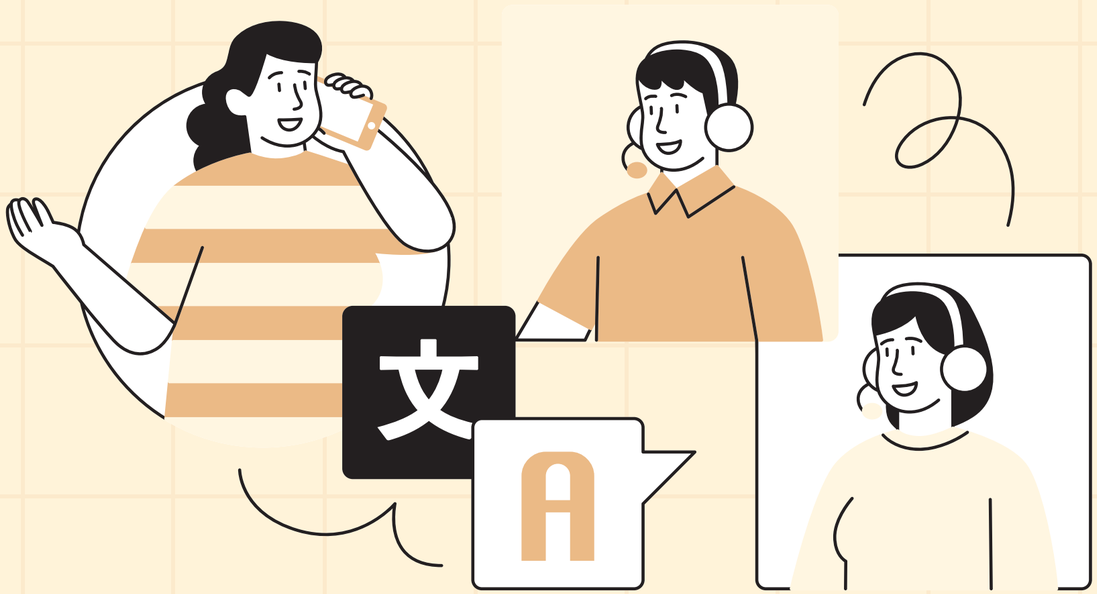
Es la manera usual en la que se organizan los compiladores.

El parser es quien dirige todo el proceso:

- Invocar al scanner.
- Invocar rutinas semanticas.

El parser es cliente del scanner.

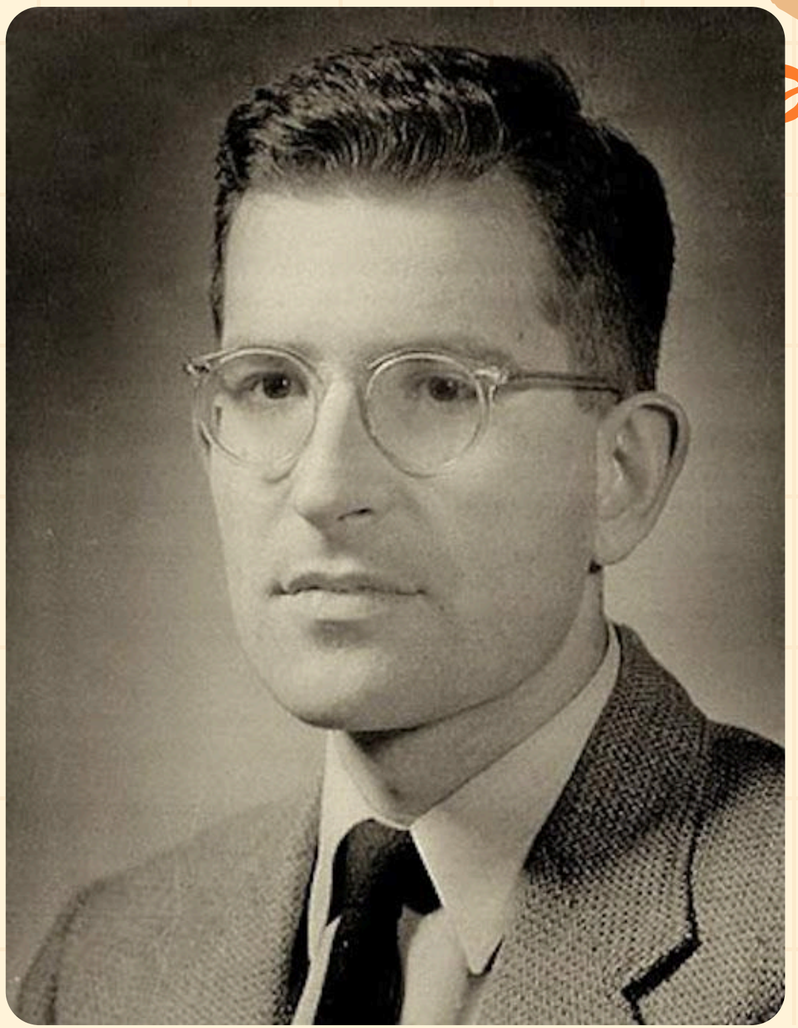
El parser podría terminar de trabajar al generar código intermedio o Lenguaje Máquina.



Avram Noam Chomsky

1928 USA

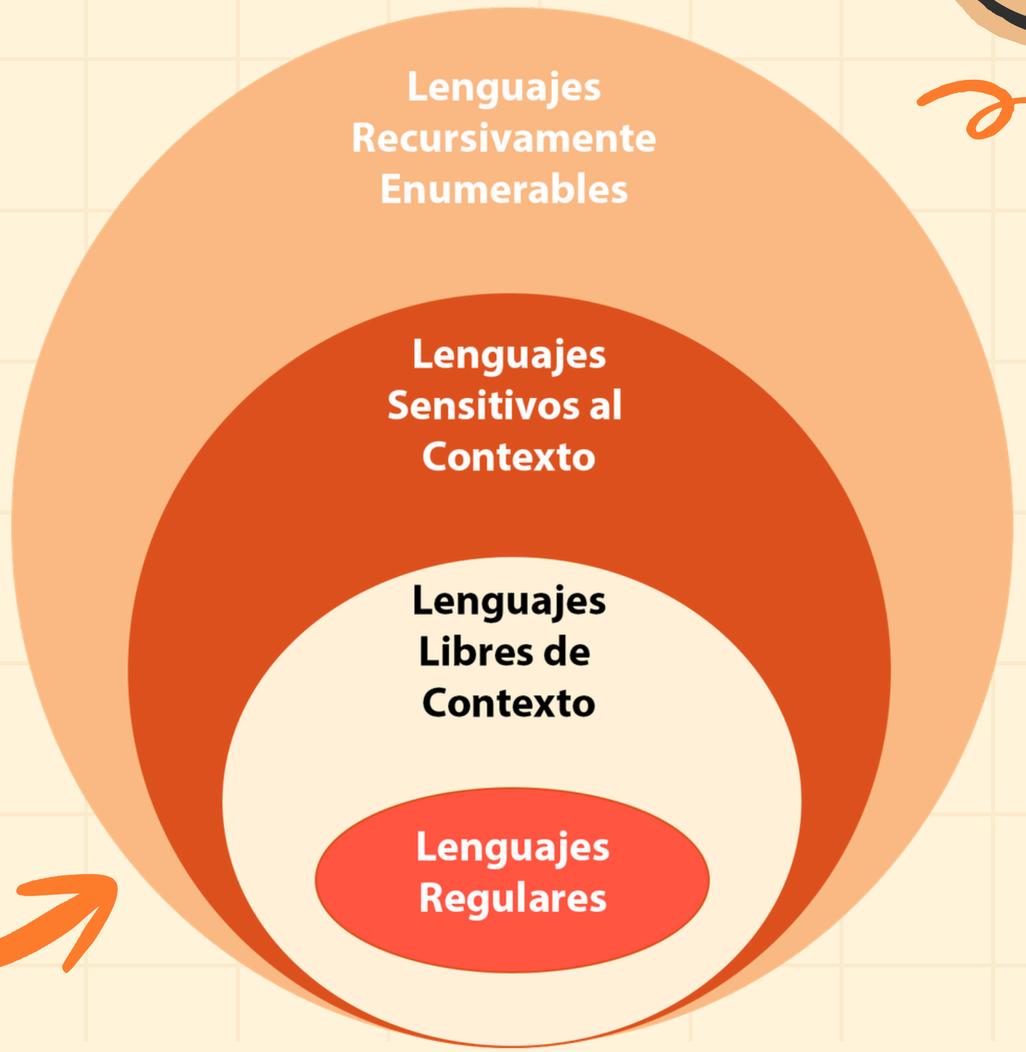
- Lingüista Filósofo, Científico Social.
- Padre de la lingüística.
- Profesor emérito del M.I.T.
- Uno de los autores más citados.
- Muy de Izquierda.



Lenguajes NO Regulares

- Gracias al Pumping Lemma podemos demostrar que existen lenguajes no regulares.
- Pero??, y estos qué?, cómo se relacionan entre si?, existe algun mecanismo que los genere o reconozca?.

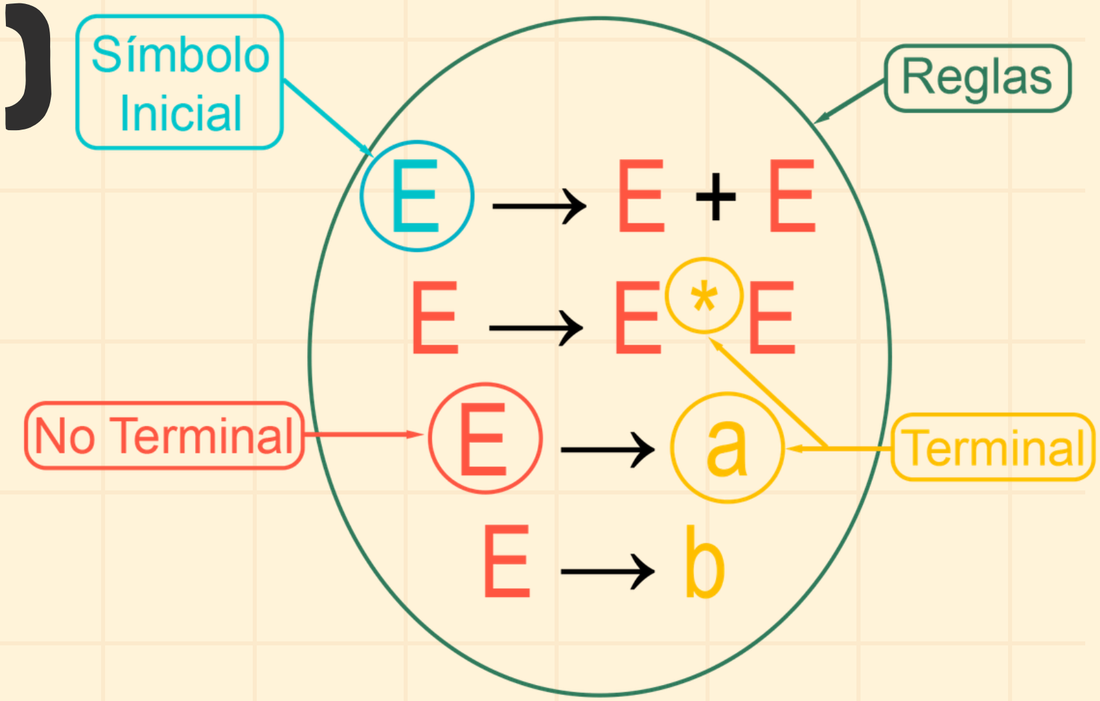
Jerarquía de Chomsky



Gramática Libre de Contexto (CFG)

- Es un cuarteto G = (V, Σ , P, S), donde:
  - V es un conjunto finito de NO Terminales.
  - Σ es un conjunto finito de Terminales.
  - P es el conjunto finito de reglas de la gramática.
  - S es el símbolo inicial.
- V ∩ Σ = ∅.
- P ⊆ V X (V ∪ Σ)*

↳ Producto Cartesiano



Los NO Terminales están a la izquierda y derecha.

Los Terminales SOLO están a la derecha.

Las Reglas tienen un ÚNICO NO Terminal a la izquierda, a la derecha puede estar ε .

Derivación

- Sean α, β, γ hileras de (V ∪ Σ)*.
  - Si se tiene una hilera w de la forma αAB y una regla de la forma A → γ entonces w se puede reescribir como α γ β.
  - La derivación se denota: w ⇒ α γ β
- Se puede derivar mientras hayan NO Terminales.
- No importa el contexto del No Terminal.

Un paso de derivación:

$$w \Rightarrow v$$

Uno o más pasos de derivación:

$$w \Rightarrow + v$$

Cero o más pasos de derivación:

$$w \Rightarrow {}^{*}v$$

Derivación de w

- Sea G una CFG y w $\in$ (V $\cup$ Σ)*
- El conjunto de hileras derivables de w se consiguen recursivamente como:
  a. w es derivable de w.
  b. Si v = αAB es derivable desde w y A $\rightarrow$ γ es una regla, entonces $\alpha\gamma\beta$ es derivable desde w.
  c. Todas y únicamente las hileras que se hicieron con el paso b, son derivables desde w.

$$w = \{ v \mid w \Rightarrow *v \}$$

Sea G:

$$E \rightarrow E + E$$
$$E \rightarrow E * E$$
$$E \rightarrow a$$
$$E \rightarrow b$$

- Hileras derivables de abba: = { abba }
- De *EaE = infinitas, { *EaE, *aaE, *aaa,...}

Definiciones

- Sea G = (V, Σ , P, S) una CFG:
  1. Una hilera w ∈ (V ∪ Σ)* es una forma sentencial de G, si y solo si S ⇒ * w.
  2. Una hilera w ∈ Σ* es una sentencia de G si y solo si S ⇒ * w.
  3. El lenguaje de G, L(G), es: {v | v ∈ Σ* y S ⇒ * w }.
  4. L(G) es el conjunto de todas las sentencias de G.

When you're studying Context-free Grammar and the professor writes this:

$$A \rightarrow \alpha \mid \beta \mid \gamma$$

